

Memory & Storage Bindings — Finals Study Guide

Everything here is one big topic really: "**When and how does a variable get its memory, and when does it lose it?**" Once that clicks, every sub-question you listed falls into place. So we go in this order:

1. Binding basics (implicit/explicit, compile-time/run-time)
 2. The 4 storage classes (static, stack-dynamic, explicit heap-dynamic, implicit heap-dynamic)
 3. Stack deep-dive
 4. Static deep-dive
 5. Heap deep-dive (incl. malloc/calloc/realloc/free, C++ new/delete)
 6. Heap disasters (dangling pointer, leak, overflow, garbage value) + how to detect them (Valgrind/LLVM)
 7. Trick-question cheat sheet
-

1. Binding Basics

Binding = deciding a property of something (type, memory location, value...) and attaching it. The question is always: **WHEN** does that decision get locked in?

Term	Meaning	Example
Explicit	You state it yourself in code	<code>int x;</code> → you explicitly said "x is an int"
Implicit	The language/compiler figures it out for you	<code>x = 5</code> in Python → nobody wrote a type, language infers <code>int</code>
Compile-time binding	Decision made before the program runs	Type of <code>int x;</code> in C is fixed at compile time
Run-time binding	Decision made while the program is executing	The <i>address</i> of a stack variable is decided only when the function is actually called

"Free-flowing framework, no type declaration" = this is just describing a **dynamically/implicitly typed language** (Python, JavaScript, Prolog). You never write a type; the type is bound to the variable **at run time**, and can even change:

```
x = 5      # x is bound to int, at runtime
x = "hello" # now rebound to str, still runtime
```

Compare with C, which is **explicit + static (compile-time) typing**:

```
int x = 5; // type explicitly declared, checked at compile time
```

Two separate axes — don't mix them up:

- **Type binding:** static (C, Java) vs. dynamic (Python, JS)
- **Storage binding:** static / stack-dynamic / heap-dynamic (covered next) — this is about *memory*, not type.

A variable can be statically typed but stack-dynamic in storage (e.g. a local `int` in C) — these are independent questions.

2. The 4 Storage Classes — the master table

This is the single most-tested table. Memorize it cold.

Storage class	When memory is allocated	When memory is freed	Who controls it	Example
Static	Before execution begins (compile/link time)	When the whole program ends	Compiler	<code>static int count;</code> global vars
Stack-dynamic	When the <i>declaration is executed</i> (function called)	When the function/block exits	Runtime system, automatic	local variables: <code>int x;</code> inside a function
Explicit heap-dynamic	Whenever an explicit allocation statement runs	Whenever an explicit deallocation statement runs (or never — that's the bug)	Programmer	<code>malloc(), new</code>

Implicit heap-dynamic	Whenever a value is <i>assigned</i> — no explicit call needed	Automatically, usually via garbage collection	Language runtime	Python lists, JS objects: <code>a = [1,2,3]</code>
------------------------------	---	---	------------------	--

Your question "**stack could be dynamic as well**" — yes: "stack-dynamic" is the official name for ordinary local variables. "Dynamic" here just means *bound at run time*, not that it lives on the heap. Don't confuse "dynamic" with "heap."

3. Stack Deep-Dive

How it works

- Every function call pushes a **stack frame** (aka activation record) containing: parameters, local variables, return address.
- When the function returns, the whole frame is popped — **automatic, fast, no programmer effort**.

```
void foo() {
    int x = 10; // stack-dynamic: allocated when foo() starts
}             // x is destroyed the instant foo() returns
```

"If a function is called/written inside main, what happens?"

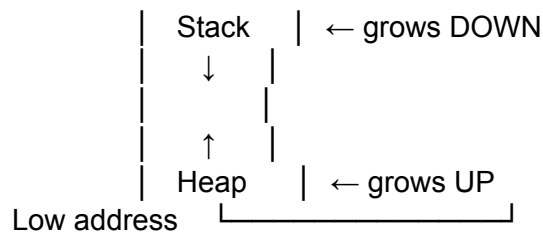
```
int main() {
    int a = 5;    // pushed onto stack when main starts
    foo();        // a NEW frame for foo() is pushed ON TOP of main's frame
    return 0;     // main's frame popped, program ends
}
```

So calling a function nested inside `main` just stacks another frame above `main`'s — that's literally why it's called a stack.

Stack grows downward / high address

- On most architectures (x86, ARM), the stack starts at a **high memory address** and grows **toward lower addresses** as more frames are pushed.
- The heap, by contrast, starts low and grows **upward**. They grow toward each other, leaving free space in the middle:

High address



This is why it's an **extendable boundary** — both regions can grow into the middle free space until they'd collide (→ stack overflow / out of memory).

Scoping & reuse of variables/identifiers

- Because each call gets a *fresh* frame, the **same identifier name** can safely be reused across different calls/scopes with no clash — this is what "scoping comes along with reusing variables" means.

```
void foo() { int x = 1; }
void bar() { int x = 2; } // different x, different frame, no conflict
```

- This is also exactly **what makes recursion possible**:

Recursion — advantage/disadvantage

- **Advantage:** each recursive call gets its own independent stack frame, so each call has its own copy of local variables/parameters. This is the "solution" that stack-dynamic allocation provides — **without stack-dynamic storage, recursion would be impossible** (all calls would fight over the same static memory).

```
int fact(int n) {
    if (n == 0) return 1;
    return n * fact(n - 1); // each call: own n, own frame
}
```

- **Disadvantage:** every call = memory + time overhead (pushing/popping frames); too much depth → **stack overflow**.

Control transfer / caller vs callee

- **Caller (called function... i.e. the function that calls)** = the function making the call.
- **Callee** = the function being called.

- When a call happens, **control transfers to the callee**: execution jumps to the callee's code, a new frame is pushed, and when the callee finishes, control returns to the caller at the point right after the call, and the callee's frame is popped.
-

4. Static Variables — Deep Dive

```
void counter() {  
    static int count = 0; // allocated ONCE, at compile/link time  
    count++;  
    printf("%d\n", count);  
}  
counter(); // prints 1  
counter(); // prints 2 <-- value PERSISTED across calls (not stack behavior!)  
counter(); // prints 3
```

Why "code is important" for static: the *initializer* (= 0) only runs **once**, the very first time — not on every call. This trips people up in exams:

```
static int x = compute(); // compute() called exactly once, ever
```

Properties to memorize:

Property	Why
Speed of execution is faster	Address is fixed at compile time — no runtime allocation cost, direct addressing
Restricts recursive subprogramming	All calls share the <i>same</i> single memory location — a recursive call would overwrite the same variable instead of getting its own copy, breaking the recursion's independence
Not deallocated until the program ends	Lives for the entire program lifetime, unlike stack (freed on return) or heap (freed on free)
Drawback	Because it must exist for the <i>whole</i> program, if you have lots of static data you permanently reserve larger memory even when it's unused most of the time — trade "ultra-fast access" for "wastes memory long-term"

5. Heap Deep-Dive

When is heap used vs stack used?

- **Stack:** size known at compile time, short lifetime tied to a function call, small/fixed data.
- **Heap:** size unknown until runtime, OR data must **outlive** the function that created it, OR data is large.

Direct vs Indirect addressing

- **Direct addressing:** the variable name refers straight to the memory holding the value (typical stack/static variable — *x* is the value's home).
- **Indirect addressing:** you access the value *through* a pointer — the variable holds an address, and you must dereference it to reach the actual data. This is exactly how heap memory is normally accessed:

```
int x = 5;      // direct: x IS the storage
int *p = malloc(sizeof(int));
*p = 5;        // indirect: p holds an ADDRESS, *p reaches the real value
```

malloc / calloc / realloc / free (C)

```
#include <stdlib.h>
```

```
// malloc: allocate raw (uninitialized) memory
int *a = (int *) malloc(5 * sizeof(int));    // 5 ints, garbage values inside
```

```
// calloc: allocate AND zero-initialize
int *b = (int *) calloc(5, sizeof(int));    // 5 ints, all set to 0
```

```
// realloc: resize a previous allocation (keeps old data if it fits)
a = (int *) realloc(a, 10 * sizeof(int));    // now 10 ints; first 5 keep old values
```

```
// free: MANDATORY once you're done, or it's a leak
free(a);
free(b);
```

Function	Initializes memory?	Use case
<code>malloc(n)</code>	No — garbage values	Fast allocation when you'll overwrite everything anyway

<code>calloc(n, size)</code>	Yes — zeroed	Safer default, slightly slower
<code>realloc(ptr, newSize)</code>	Preserves old data, new part uninitialized	Growing/shrinking an existing array
<code>free(ptr)</code>	—	Releases heap memory back to the system

C++ heap allocation

```
int *p = new int(5);    // allocate + initialize to 5
delete p;              // free ONE object
```

```
int *arr = new int[10]; // allocate array
delete[] arr;           // MUST use delete[] for arrays, not delete
```

"When does heap need to be freed?" / "user-allocated role"

Any time **you** (the programmer) explicitly requested memory — `malloc`, `calloc`, `realloc`, `new` — **you** are responsible for releasing it with `free/delete` once it's no longer needed, and specifically **before you lose your last pointer to it** (otherwise it becomes unreachable → leak, see below).

"When is there no canonical form of the heap?"

Unlike the stack (which always follows a strict, predictable Last-In-First-Out order), the heap has **no fixed/standard ("canonical") layout**. Blocks are allocated and freed in arbitrary order and sizes, chosen by whatever allocation algorithm the runtime uses (first-fit, best-fit, etc.), so the heap's internal shape is unpredictable and becomes **fragmented** over time — free and used blocks scattered irregularly, unlike the stack's clean, orderly growth.

Heap overflow — how it happens

Writing **past the boundary of an allocated heap block**, corrupting adjacent memory:

```
int *arr = malloc(5 * sizeof(int));
arr[10] = 1; // OUT OF BOUNDS — heap overflow, undefined behavior, memory corruption
```

6. Heap Disasters — reproduce each in code

(a) Garbage value (uninitialized variable)

```
int *p = malloc(sizeof(int));
printf("%d\n", *p); // GARBAGE — malloc doesn't zero memory, contains leftover bits
```

Fix: use `calloc`, or explicitly `*p = 0;`.

(b) Dangling pointer (didn't release properly / used after free)

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);
printf("%d\n", *p); // DANGLING POINTER — p points to freed memory, undefined behavior
```

A pointer becomes "dangling" when it still holds an address, but the memory at that address has already been freed (or, similarly, when it points to a stack variable that has gone out of scope).

Fix: `p = NULL;` right after `free(p);` so accidental reuse crashes cleanly instead of silently corrupting data.

"What happens with a dangling pointer when `return 0` runs at the end of `main`?"

`return 0;` just ends the program — the **OS reclaims ALL memory** (stack, heap, everything) regardless of any dangling pointers or un-freed blocks that existed. Nothing "bad" happens *at exit* purely because of a dangling pointer — the danger is only if you **dereference** it *before* that point. (Still bad practice / real leaks in a long-running program, but exiting the process is a full memory reset by the OS.)

(c) Memory leak (didn't release / lost the pointer)

```
void leak() {
    int *p = malloc(sizeof(int));
    *p = 5;
    // function ends WITHOUT free(p)
    // p (the only reference) is destroyed when leak() returns
    // → the heap block is now unreachable forever = LEAK
}
```

Leak = memory that's still allocated but **no pointer refers to it anymore**, so it can never be freed for the rest of the program's life.

7. Detecting Leaks — Valgrind & LLVM

Valgrind (Linux, works on compiled C/C++ binaries)

```
gcc -g program.c -o program      # compile with debug symbols
valgrind --leak-check=full ./program
```

`--leak-check=full` gives a detailed report of every leaked block and where it was allocated.

LLVM / Clang tools (AddressSanitizer & LeakSanitizer)

```
clang -fsanitize=address -g program.c -o program
./program
```

AddressSanitizer (ASan) catches heap overflows, use-after-free (dangling pointers), and double-frees. LeakSanitizer (bundled with ASan, or standalone via `-fsanitize=leak`) specifically reports memory leaks at program exit.

On macOS, the equivalent built-in tool is:

```
leaks --atExit -- ./program
```

8. Cheat Sheet — Trick-Question Traps

Trap	Correct answer
"Dynamic" always means heap	False — stack-dynamic exists; "dynamic" = bound at run time, not "lives on heap"
Static variable resets every call	False — initializer runs once ever; value persists
<code>malloc</code> zeroes memory	False — garbage values; <code>calloc</code> zeroes
Stack grows upward	False (on typical architectures) — grows toward lower addresses
Recursion works fine with static/global vars for locals	False — all calls share one memory slot, breaking independent recursive state
Freeing solves everything, forget the pointer after	Risk of dangling pointer — always set pointer to <code>NULL</code> after <code>free</code>

Heap memory is freed automatically like stack

False for explicit heap-dynamic (C/C++) — programmer must free; **True** only for implicit heap-dynamic (Python/JS garbage collection)

Program crash guaranteed if you exit with unfreed heap memory

False — OS reclaims everything on process exit; the problem is leaks *during* a running/long-lived program, not at exit

Memory & Storage Bindings — Q&A Practice Set

Organized in four tiers: **Basic** → **Intermediate** → **Tricky/Conceptual** → **Quick-Fire (time-crunch)**. Companion to [memory-storage-study-guide.md](#) — read that first if any answer here feels too compressed.

Tier 1 — Basic (should answer in seconds)

Q1. What is binding? A: Deciding a property of a variable (type, memory location, value) and attaching it to that variable.

Q2. What's the difference between implicit and explicit binding? A: Explicit = you state it in code (`int x;`). Implicit = the language infers it for you (`x = 5` with no declared type).

Q3. Name the four storage classes. A: Static, stack-dynamic, explicit heap-dynamic, implicit heap-dynamic.

Q4. Which storage class do ordinary local variables in C belong to? A: Stack-dynamic — allocated when the function is called, freed when it returns.

Q5. Which storage class does `malloc()` create? A: Explicit heap-dynamic — programmer explicitly allocates (and must explicitly free).

Q6. Which direction does the stack grow? A: Downward — from high addresses toward low addresses.

Q7. What does `calloc` do that `malloc` doesn't? A: Zero-initializes the allocated memory; `malloc` leaves it as garbage values.

Q8. What must you always pair with `malloc/calloc/realloc`? A: `free()` — otherwise you get a memory leak.

Q9. What's a dangling pointer? A: A pointer that still holds an address, but the memory at that address has already been freed (or gone out of scope).

Q10. Is a static variable re-initialized every time the function runs? A: No — its initializer runs exactly once, ever; the value persists across calls.

Tier 2 — Intermediate

Q11. Why is a static variable's initialization code important? A: Because it only executes on the *first* call — if it calls a function (`static int x = compute();`), that function runs exactly once for the whole program, not on every call.

Q12. Why is static variable access faster than heap access? A: Its address is fixed at compile time (direct addressing) — no runtime allocation/lookup cost, unlike heap access which usually goes through a pointer (indirect addressing).

Q13. Why do static variables restrict recursive subprogramming? A: All calls to the function share the *same single* memory location for that variable — a recursive call would overwrite the same slot instead of getting an independent copy, breaking the recursion.

Q14. What's the main drawback of static allocation, despite being fast? A: The memory is reserved for the *entire* program lifetime, even when unused most of the time — trades speed for permanently larger memory footprint.

Q15. What's the difference between direct and indirect addressing? A: Direct: the variable name refers straight to where the value lives (`int x = 5;`). Indirect: you go through a pointer that holds an address, then dereference it (`int *p; *p = 5;`).

Q16. Explain caller vs callee. A: Caller = the function making the call. Callee = the function being called. Control transfers to the callee when the call happens, and returns to the caller once the callee finishes.

Q17. Why does stack-dynamic allocation make recursion possible? A: Every call gets its own fresh stack frame, so each recursive call has independent copies of its local variables/parameters — no shared state to conflict.

Q18. What's a disadvantage of recursion despite the stack solving the "shared state" problem? A: Memory + time overhead per call (pushing/popping frames); too much recursion depth causes a **stack overflow**.

Q19. When should you use heap instead of stack? A: When the size isn't known until runtime, when data must outlive the function that created it, or when the data is too large for the stack.

Q20. What does `realloc` preserve? A: The original data (up to the smaller of old/new size) when resizing a previous allocation; any newly added space is uninitialized.

Q21. In C++, what's the difference between `delete` and `delete[]`? A: `delete` frees a single object (`new int(5)`); `delete[]` frees an array (`new int[10]`) — using the wrong one is undefined behavior.

Q22. What causes heap overflow? A: Writing past the boundary of an allocated heap block, e.g. `arr[10] = 1;` on a `malloc(5*sizeof(int))` block — corrupts adjacent memory.

Tier 3 — Tricky / Conceptual (the ones people lose marks on)

Q23. "Dynamic" always means the variable lives on the heap." True or false? A: **False.** "Dynamic" means *bound at run time*. Stack-dynamic variables are bound at run time too but live on the stack, not the heap.

Q24. If a function is called inside `main`, what exactly happens to memory? A: A new stack frame for that function is pushed on top of `main`'s existing frame. When it returns, only that frame is popped — `main`'s frame is untouched underneath.

Q25. Why is the heap said to have "no canonical form"? A: Unlike the stack's strict, predictable Last-In-First-Out order, heap blocks are allocated/freed in arbitrary order and sizes by whatever algorithm the allocator uses — so its internal layout is unpredictable and becomes fragmented, with no fixed/standard shape.

Q26. A program has a dangling pointer, and then `return 0;` executes at the end of `main`. What actually happens? A: The OS reclaims **all** process memory on exit — stack, heap, everything — regardless of the dangling pointer. Nothing bad happens *purely from exiting*; the danger only exists if the dangling pointer is **dereferenced before** that point.

Q27. When exactly does heap memory "need" to be freed? A: As soon as it's no longer needed, and critically **before you lose your last pointer/reference to it** — once the last reference is gone, the block becomes permanently unreachable (a leak).

Q28. A variable is declared but never initialized, and you print it. What do you see, and why? A: A **garbage value** — leftover bits from whatever previously occupied that memory

address. `malloc` (and uninitialized stack locals) do NOT auto-zero memory; only `calloc` (or explicit assignment) guarantees zero.

Q29. Stack-dynamic variables and implicit heap-dynamic variables are both "dynamic" — what's actually different about them? A: Stack-dynamic: allocated/freed automatically tied to a function call's lifetime, lives on the stack. Implicit heap-dynamic: allocated on the heap the moment a value is assigned, with **no explicit allocation call**, and freed automatically (e.g. by garbage collection) — think Python's `a = [1, 2, 3]`.

Q30. Why can the same identifier name (like `x`) be reused safely across different function scopes without conflict? A: Each function call gets a fresh, independent stack frame — the `x` in one frame is a physically different memory location from the `x` in another, even with identical names.

Q31. What's the difference between a memory leak and a dangling pointer? (Common confusion!) A: **Leak** = memory is still allocated, but no pointer refers to it anymore (unreachable, can't be freed). **Dangling pointer** = the memory *has been freed* (or gone out of scope), but a pointer still holds its old address and could be wrongly dereferenced.

Q32. Why does explicit heap-dynamic storage exist as a *separate* category from implicit heap-dynamic, if both live on the heap? A: The distinguishing factor is **who/what triggers allocation and deallocation** — explicit needs a programmer-written call (`malloc/new ... free/delete`); implicit happens automatically via assignment and garbage collection, with zero explicit calls from the programmer.

Q33. Is type binding (static/dynamic typing) the same axis as storage binding (static/stack/heap)? Explain with an example that shows they're independent. A: No — they're independent axes. Example: a local `int` in C is **statically typed** (type fixed at compile time) but **stack-dynamic** in storage (memory bound at run time, freed on return). You can have any combination.

Q34. Why does `calloc` being "slightly slower" than `malloc` matter as a design trade-off? A: `calloc` must zero every byte of the allocated block before returning it — extra work `malloc` skips. Use `malloc` when you'll immediately overwrite all the memory anyway (skip redundant zeroing); use `calloc` when uninitialized garbage would be dangerous (e.g. reading before writing).

Q35. If a static variable and a stack variable both have the exact same name in the same function, which "wins," and why does this matter for the "code is important" idea? A: (Trick — in real C this would be a redeclaration error, but conceptually:) the point being tested is that a `static` declaration is only textually inside the function, but its **storage lifetime and identity are global** — it is NOT re-created per call like a stack local would be, even though it's scoped like one. Scope ≠ lifetime; don't conflate the two.

Tier 4 — Quick-Fire (time-crunch format — answer in one line each)

Q36. Heap grows which direction relative to stack? → **Upward, toward the stack** (they grow toward each other).

Q37. Fastest storage class to access? → **Static** (fixed address, direct addressing).

Q38. Slowest / most manually managed storage class? → **Explicit heap-dynamic.**

Q39. Which storage class is freed automatically via garbage collection? → **Implicit heap-dynamic.**

Q40. Tool to detect memory leaks on Linux? → **Valgrind** (`valgrind --leak-check=full`).

Q41. LLVM/Clang flag to catch use-after-free and overflows? → `-fsanitize=address` (AddressSanitizer).

Q42. LLVM/Clang flag focused specifically on leaks? → `-fsanitize=leak`.

Q43. macOS built-in leak-checking command? → `leaks --atExit -- ./program`.

Q44. What should you do to a pointer immediately after `free(p)`? → `p = NULL;`

Q45. What does indirect addressing require before use? → **Dereferencing** (`*p`).

Q46. What allocates memory but does NOT initialize it? → `malloc`.

Q47. What allocates AND zero-initializes? → `calloc`.

Q48. What resizes an existing heap block? → `realloc`.

Q49. In C++, allocate a single int on the heap? → `new int(5);`

Q50. In C++, free an array allocated with `new[]`? → `delete[] arr;`

Q51. Why can't static storage support independent recursive calls? → **One shared memory slot for all calls, not one per call.**

Q52. What's pushed onto the stack on every function call? → **A stack frame / activation record** (params, locals, return address).

Q53. What happens to a stack frame when its function returns? → **It's popped/destroyed automatically.**

Q54. Does exiting a program via `return 0;` clean up heap memory even if you forgot to `free`? → **Yes — the OS reclaims everything on process exit.**

Q55. Writing `arr[10]` on a `malloc(5*sizeof(int))` array — what is this called? → **Heap overflow / out-of-bounds write.**